1).Write a program to create a robot (i) with gear (ii) without gear and move it forward, left, and right.

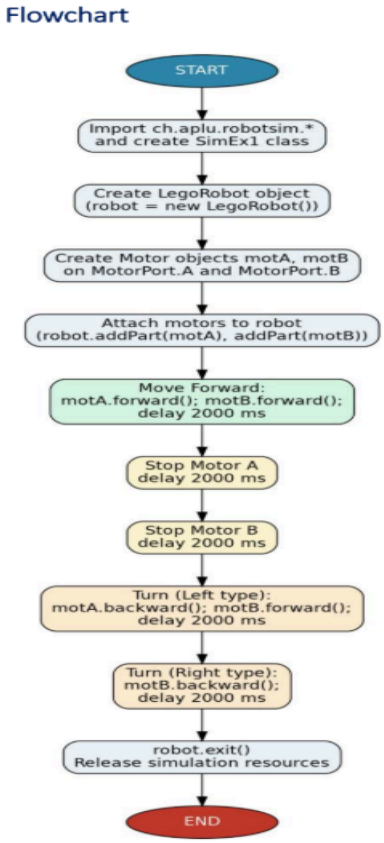
Theory
Robotics involves designing and controlling autonomous or semi-autonomous mobile machines using software and actuators. Robotsim and JGameGrid provide a safe simulation environment to study kinematics, command sequencing, and differential-drive motion without physical hardware.
*JGameGrid: A 2D Java engine handling actors, animation, timers, and collision detection.
*Robotsim: A simulation library built on JGameGrid that provides robot and mechanical gear classes.
*Java IDE: An environment to compile and execute Java applications using external JARs.
*LegoRobot: The base robot entity in Robotsim used to attach parts like motors, gears, and sensors.
*Procedure
1.Configure JDK and add JGameGrid.jar and Robotsim.jar to the project build path.
2.Code RobotWithoutGear.java and RobotWithGear.java, then compile and run them.
3.Observe and compare basic vs. gear-driven forward, turning, and stop behaviors in the simulator.
*Algorithm
1.Start program and instantiate a LegoRobot.
2.Instantiate a Gear object and attach it to the robot.
3.Move forward, wait, and turn left.
4.Move forward, execute a left arc, move forward, and execute a right arc.
5.Exit simulation and end program.
*Conclusion
Demonstrated direct and gear-driven mobile robot control using Robotsim and JGameGrid to execute coordinated linear and arc motions on a 2D plane.

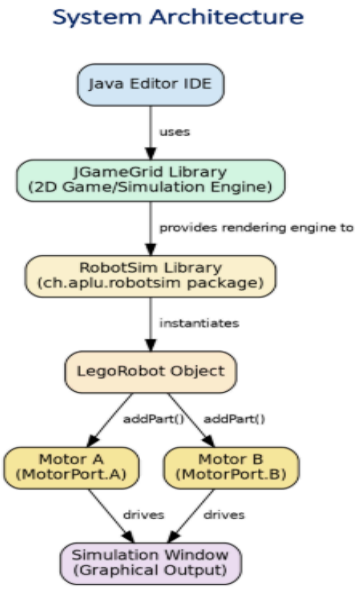
2).Write a program to create a robot with two motors and move it forward, left, and right.

*Theory
Mobile robot and Locomotion
A mobile robot io a machine capable of autonomous or programmed movement within its environment The most common locomotion mechanism used in low - cost and educational roboto is the wheeled differential drive System in this configuration, the robot has two independently powered wheels, usually mounted on a comman axis,along with one or more passive caster or ball wheels for balance.
Differential drive mobile robots navigate using two independently driven coaxial wheels and passive casters for balance, steering via wheel speed variation rather than mechanical linkages.
Libraries used:
1.LegoRobot: Robot chassis class that opens the 2D simulation canvas for attaching hardware components.

2.Motor: Controls individual DC motors attached to ports using methods like forward(), backward(), and stop().
3.MotorPort: Enumeration specifying physical controller ports (e.g., MotorPort.A, MotorPort.C).
4.Tools: Provides execution controls such as Tools.delay() to render observable movements.

Conclusion
Simulated a two-motor differential-drive robot using RobotSim and JGameGrid, successfully executing forward motion, stops, and directional turns via independent motor control.





3).Write a program to do a square using a while loop, doing steps with a for loop, changing directions based on a condition, and controlling motor speed using switch case.

*Theory:
-RobotSim and the TurtleRobot Class:
RobotSim is a Java-based robot simulation framework that lets students test robot motion algorithms without needing physical hardware. The ch.aplu.robotsim package provides ready-made classes such as TurtleRobot, which simulates a small robot that can move forward, turn left or right, and change its speed. Commands such as forward(), right(), left(), setTurtleSpeed(), and exit() are used to control the simulated robot step by step and the

resulting path is displayed on screen. This makes it a convenient tool for learning core programming constructs in the context of robot motion planning before applying the same logic to real robots.

-Control Structures used in Robot Programming:

while loop: Repeats code while a condition is true, used here to trace a 4-sided square.

for loop: A compact loop with counter handling, used here to repeat step patterns 4 times.

if-else statement: Selects between two paths conditionally, used here for random turn decisions.

switch case statement: Evaluates a variable against multiple values, used here to set motor speeds.

*Algorithm:

1. Squaring using while loop:

Start the program and create a TurtleRobot object.

Initialize the loop counter i to 0.

Check the condition i < 4; if true, continue to step 4, otherwise go to step 6.

Move the robot forward by 50 units, turn it right by 90 degrees, and increment i by 1.

Repeat from step 3.

Call robot.exit() to close the simulation, and stop the program.

2. Steps using For loop:

Start the program and create a TurtleRobot object.

Set up a for loop with i = 0, condition i < 4, and increment i after each pass.

In each pass of the loop, move the robot forward 20 units, turn right 90 degree, move forward 20 units again, and turn left 90 degrees.

Repeat the loop until the condition i < 4 becomes fail.

Call robot.exit() to close the simulation, and stop the program.

3. Changing Direction based on Condition:

Start the program and create a TurtleRobot object.

Initialize the loop counter i to 0 and repeat the following steps while i < 7:

Generate a random number a between 0 and 1 using Math.random().

Check if a \le 0.5. If true, turn the robot right by 90 degrees; otherwise turn it left by 90 degrees.

Move the robot forward by 100 units and increment i by 1.

Repeat from step 3 until i reaches 7.

4. Controlling Motor Speed using Switch case:

Start the program and create a TurtleRobot object.

Initialize the loop counter k to 1 and repeat the following steps while k < 6:

Use a switch statement on k to select a speed value: 20 for k=1, 40 for k=2, 60 for k=3, 80 for k=4, and a default speed of 50 for any other value.

Set the robot's speed using robot.setTurtleSpeed(speed).

Move the robot forward by 200 units and turn it right by 90 degrees.

Increment k by 1 and repeat from step 3 until k reaches 6.

Call robot.exit() to close the simulation and stop the program.

Conclusion:

This practical demonstrated the use of while, for, if-else, and switch case statements to control a TurtleRobot in RobotSim. It helped in understanding how Java control structure are used to manage robot movement, decision-making and repetitive actions.

4).Write a program to create a robot with light sensors to follow a line.

-Theory

A line-following robot uses a downward-facing light sensor to autonomously follow a track by distinguishing between high reflectance (bright floor) and low reflectance (dark line). Based on these intensity readings, event-driven steering corrections maintain tracking:

Bright Surface: Triggers bright() to steer slightly toward the line using gear.rightArc().

Dark Line: Triggers dark() to steer back using gear.leftArc().

Continuous alternating adjustments between both states keep the robot aligned with the line edge via LightListener events.

-Algorithm

1.Start the program and import required RobotSim, JGameGrid, and AWT libraries.

2.Create a class implementing the LightListener interface.

3.In the constructor, create a LegoRobot object and a Gear object for movement.

4.Create a LightSensor connected to port S3 and attach both the Gear and sensor to the robot.

5.Register the LightListener with the sensor to catch bright/dark events.

6.Draw the circular track in the background using fillArc() inside the _init() method.

7.Move the robot forward using gear.forward().

8.When bright surface is detected, call bright() and turn right using gear.rightArc().

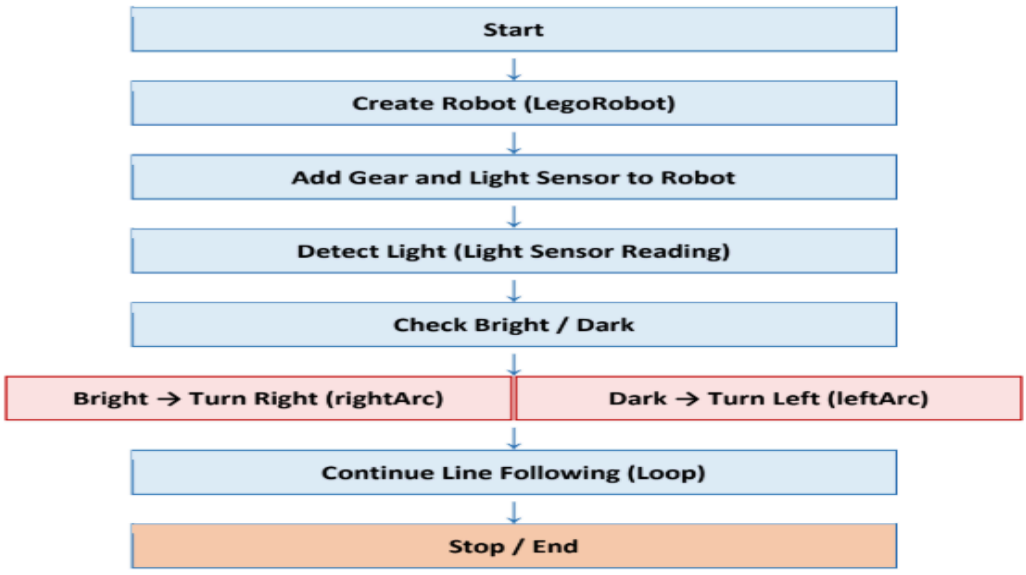
9.When dark area is detected, call dark() and turn left using gear.leftArc().

10.In main(), start the simulation to follow the line continuously, then stop execution when done.

-Conclusion

A Java program was executed to simulate an autonomous line-following robot using RobotSim and JGameGrid. The practical demonstrated how dynamic LightSensor threshold readings and LightListener callbacks (bright()/dark()) control steering arcs to achieve smooth path tracking.

Flowchart



5).Write a program to create a robot that does a circle using 2 motors.

-Theory

A DC motor converts electrical energy into rotational motion to drive robot wheels. In a two-wheeled differential drive configuration, independent motors are attached to MotorPort.A and MotorPort.B.

Straight Motion: Occurs when both motors run forward at identical speeds.

Circular Motion: Occurs when different speed values are assigned to each motor. The wheel spinning faster covers more ground than the slower wheel, forcing the robot to curve toward the slower side continuously and trace a circular path.

-Algorithm

1.Start the program.

2.Instantiate a LegoRobot object to represent the chassis.

3.Instantiate two Motor objects connected to MotorPort.A and MotorPort.B.

4.Attach both motors to the robot using addPart().

5.Set unequal speed values for Motor A and Motor B.

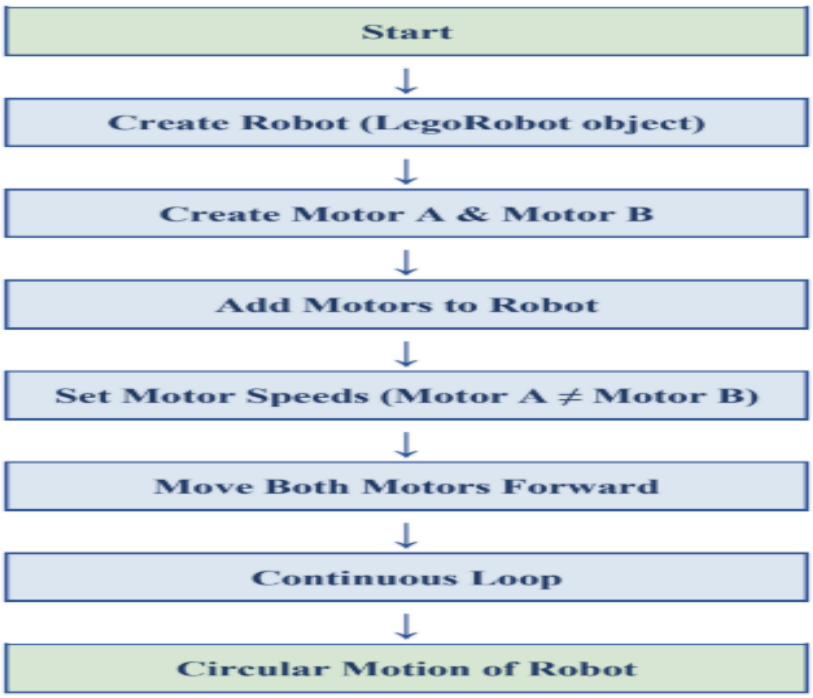
6.Drive both motors using the forward() method.

7.Run the forward motion continuously in a loop so the robot traces a circular trajectory.

8.Stop and exit the program.

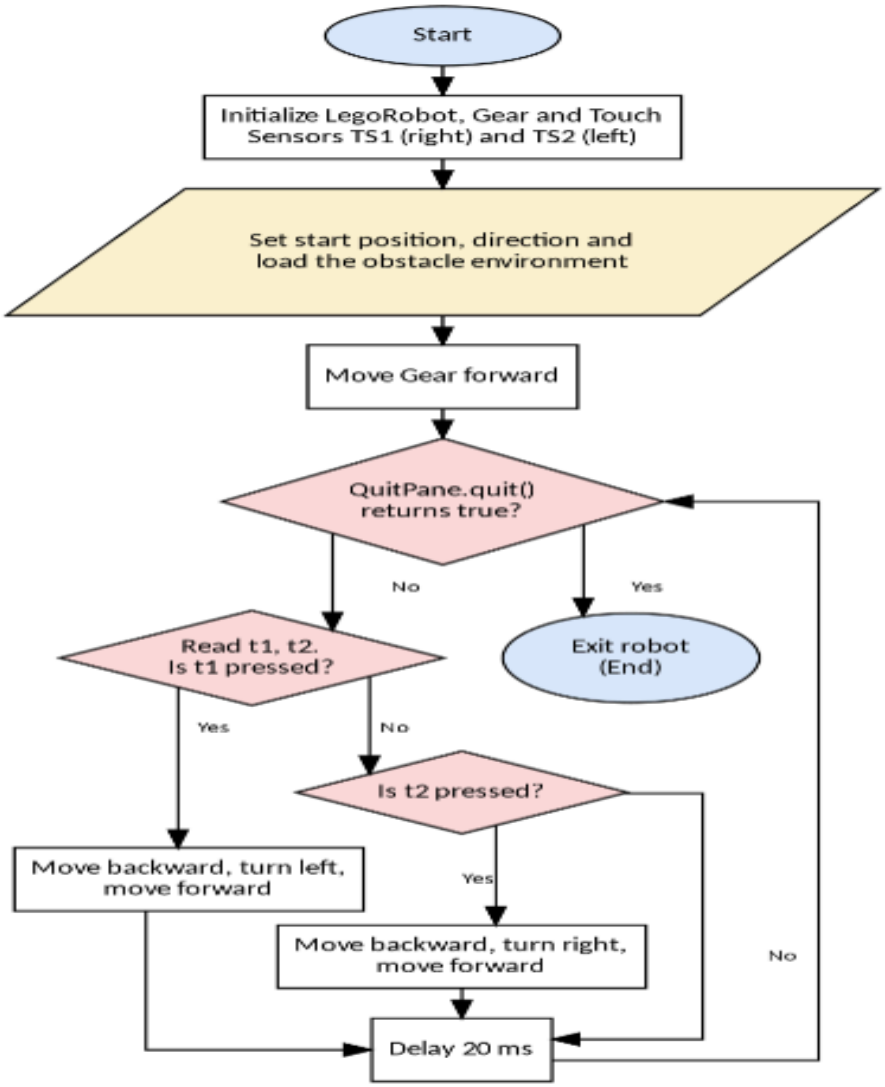
-Conclusion

The simulation successfully demonstrated circular locomotion using two independent motors in RobotSim. Setting unequal wheel speeds verified that differential velocity dictates turn curvature and motion path geometry.



6).Write a program to create a path-following robot.

-Theory
A path-following robot navigates a course and avoids obstacles using reactive collision sensing. Using RobotSim, a LegoRobot is equipped with a Gear for directional drive and two bumper touch sensors (TS1 right, TS2 left). It polls isPressed() inside a loop to steer around obstacles without a pre-mapped path:
Right/Both Pressed: Reverses and turns left.
Left Pressed: Reverses and turns right.
Clear: Continues forward.
RobotContext defines starting positions and obstacle layouts, while QuitPane.quit() and robot.exit() manage simulation shutdown.
-Algorithm
1.Start program, create LegoRobot, and attach a Gear.
2.Attach touch sensors TS1 (right, S1) and TS2 (left, S2).
3.Set grid position, direction, and obstacles via RobotContext.
4.Move forward; loop until QuitPane.quit() is true:
5.Read t_1 (TS1.isPressed()) and t_2 (TS2.isPressed()).
a.If t_1 is true: reverse, turn left, move forward.
b.Else if t_2 is true: reverse, turn right, move forward.
c.Else: move forward; add a 20 ms delay.
6.On quit, call robot.exit() and stop program.
-Conclusion
A path-following robot was successfully simulated in Java using RobotSim, demonstrating reactive obstacle avoidance using dual touch sensors.



7). Write a program to register obstacles.

-Theory

RoboSim simulates a LEGO Mindstorms robot using LegoRobot, where locomotion (Gear) and collision detection (TouchSensor) components are attached before execution.

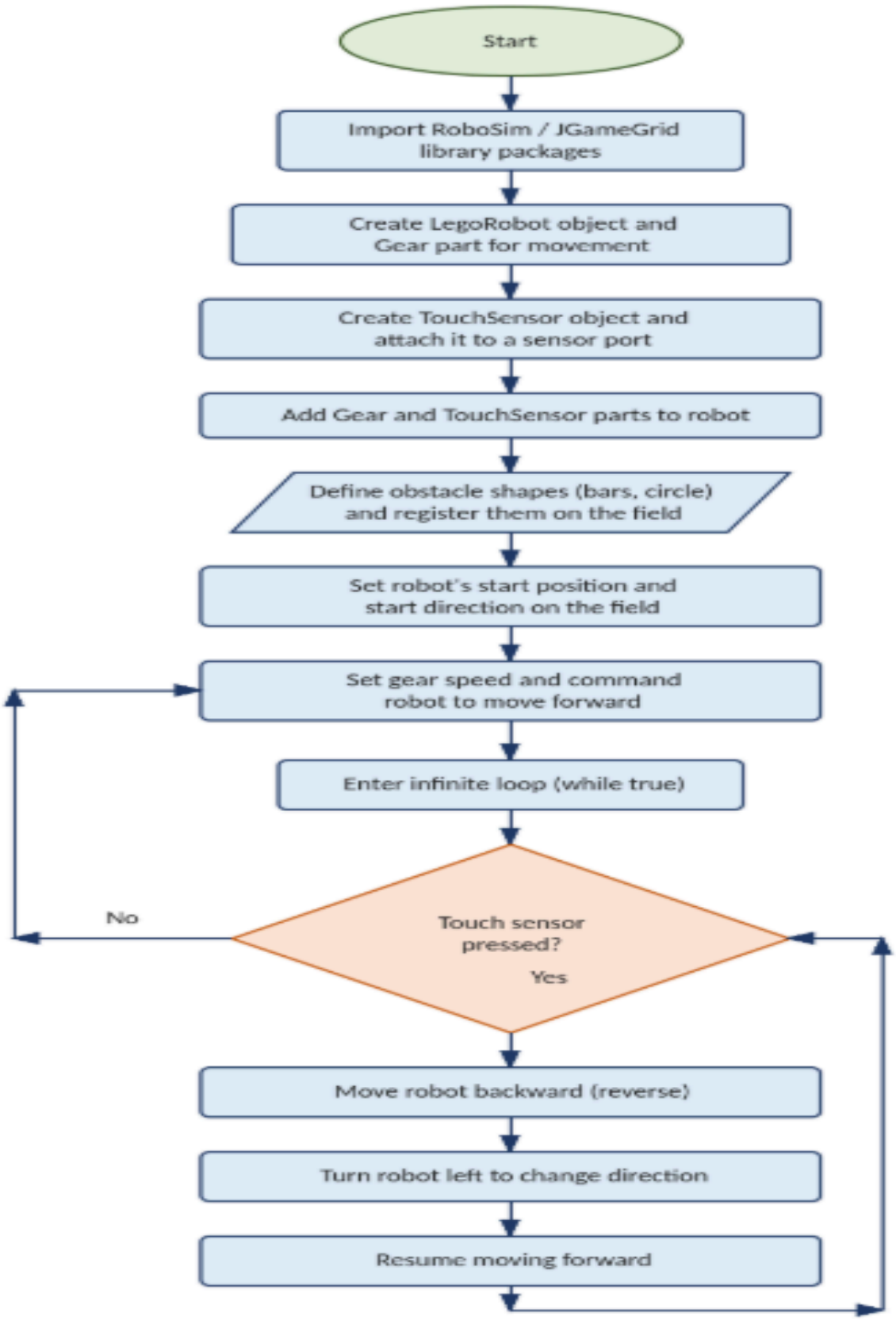
The RobotContext class configures the simulation environment by defining starting coordinates, heading, navigation controls, and registered obstacles using useObstacle(image, x, y).

Obstacles are created as bitmap objects (GGBitmap)—such as colored rectangular bars and circles—and placed at designated field coordinates. During motion (gear.forward()), an infinite polling loop checks touchSensor.isPressed():

Collision Detected: The robot executes an evasive maneuver (reverses, turns left by a fixed angle, and resumes forward motion) to navigate around the obstacle.

Clear Path: Forward movement continues uninterrupted.

-Algorithm
1.Import RoboSim, JGameGrid, and java.awt packages.
2.Instantiate LegoRobot, attach a Gear object, and attach a TouchSensor to a designated sensor port.
3.Define helper methods to render colored geometric obstacle shapes (rectangular bars and circles).
4.Using RobotContext, configure initial robot position, direction, navigation bar, and register obstacles at fixed coordinates via useObstacle().
5.Set gear speed and command the robot to move forward.
6.Enter an infinite loop to monitor motion:
7.Poll sensor state via isPressed().
8.If pressed, reverse a short distance, turn left by a fixed angle, and resume forward motion.
9.If not pressed, continue moving forward.
10.Repeat continuous collision monitoring across the simulation run.
-Conclusion
The program successfully registered multiple colored obstacles of various shapes on the RoboSim simulation field and used a TouchSensor to detect collisions and execute evasive maneuvers.


Start
Import RoboSim / JGameGrid library packages
Create LegoRobot object and Gear part for movement
Create TouchSensor object and attach it to a sensor port
Add Gear and TouchSensor parts to robot
Define obstacle shapes (bars, circle) and register them on the field
Set robot's start position and start direction on the field
Set gear speed and command robot to move forward
Enter infinite loop (while true)
Touch sensor pressed?
No
Yes
Move robot backward (reverse)
Turn robot left to change direction
Resume moving forward